A.1 TRAINING DETAILS

We provide the code of our methods and hyper-parameters configuration files in `https://github.com/shoot-tree-search/sts`.

The training loop follows the logic of Algorithm 2. We use a distributed setup with 30 workers and a replay buffer of size 30000. We perform 1000 optimizer updates on batches of transitions whenever all workers collect and store one full episode. During batch sampling, we ensured an equal amount of examples from solved and unsolved episodes. In GRF and Sokoban experiments, each episode was limited to 100 and 200 time steps, respectively.

A value function approximator, $\mathbf{V}_\theta$, is trained via the MSE loss using targets calculated by `CALCULATE_TARGET`. In shooting experiments we use "reward-to-go" targets $\sum_{i=t+1}^{T} \gamma^{i-t-1} r_i$, where T is the terminal time-step in an episode. In MCTS and STS in GRF experiments (see Section A.5 for details) we use "tree action-values" targets, similar to the one used in Hamrick et al. (2020); Miłoś et al. (2019).

Policy, π_ϕ, is trained using the cross-entropy loss. As targets, we use one-hot encoded actions chosen in the environment for Random Shooting and the empirical distribution of actions chosen in the root during the planning for Bandit Shooting, MCTS, and STS.

The total loss is a weighted sum of the value function (or the Q-function) loss, the policy loss (weighted by $1\mathrm{e}{-2}$ in Random Shooting and Bandit Shooting, and $1\mathrm{e}{-3}$ in MCTS and STS), and a regularizing, l_2 term (weighted by $1\mathrm{e}{-6}$).

A pre-trained PPO policy in Shooting methods was obtained using a script included in the Google Research Football repository (see Kurach et al. (2019)) and the OpenAI Baselines (Dhariwal et al. (2017)) PPO2 implementation.

A.2 HYPER-PARAMETERS

Table 3 presents hyper-parameters used in our experiments. These were based on hyper-parameters previously proposed in the literature and substantial amount of tuning experiments (> 3000).

A.3 NETWORK ARCHITECTURES

In GRF experiments we use two different state representations: 'simple115' and 'extended' (see Section A.8). In the former case, we use an MLP architecture with two hidden layers of 64 neurons, while in the latter case, we use 4 convolutional layers with 16, 3x3, filters, zero-padding and stride 2, followed by a dense layer of 64 neurons. In both cases, two heads, corresponding to a value function (or Q-function for MCTS and STS) and policy, follow.

In Sokoban experiments, we use 5 convolutional layers of 64, 3x3, filters with zero-padding and stride 1, followed by a dense layer of 128 neurons and heads corresponding to a value function and policy (policy is used only for Shooting methods).

In all the cases, we use the ReLU non-linearity. We use the standard Keras initialization schemes, except for MCTS and STS in GRF experiments, see Section A.8.2.

Parameter	Sokoban Shooting	Sokoban MCTS	Sokoban STS	Google Research Football Shooting	Google Research Football MCTS	Google Research Football STS
Number of passes C	48	50	10	30	300	30
Planning horizon H	5	1	5	10	1	10
Discounting γ	0.99	0.99	0.99	0.95 / 0.99[1]	0.99	0.99
Exploration weight c_{puct}	10.0[2]	0.0	0.0	1.0 / 2.5[3]	1.0	1.0
Policy π_ϕ temperature[4]	2.0	-	-	2.0	1.0	1.0
Action sampling temp. τ	-	-	-	0.3[5]	0.3	0.3
Dirichlet parameter α	-	-	-	0.03[5]	0.3	0.3
Noise weight c_{noise}	-	-	-	0.1[5]	0.1	0.1 / 0.3[6]
Depth limit `depth limit`[7]	-	-	-	-	30	30
VF zero-initialization[8]	no	no	no	no	yes	yes
Optimizer	RMS	RMS	RMS	RMS	Adam	Adam
Learning rate	2.5e−4	2.5e−4	2.5e−4	1.0e−4	1.0e−3	1.0e−3
Batch size	32	32	32	64	64	64
Target function[9]	*rew2goT*	*treeT*	*treeT*	*rew2goT*	*treeT*	*treeT*

[1] All $\gamma = 0.99$ except for Shooting experiments with a uniform and a pre-trained PPO policy, where $\gamma = 0.95$.
[2] Applies only to Bandit Shooting.
[3] $c_{puct} = 1.0$ for Bandit Shooting with a uniform and a pre-trained PPO policy and $c_{puct} = 2.5$ for Bandit Shooting with a trained policy.
[4] Softmax temperature. MCTS and STS in Sokoban does not use policy, see Section A.5 for details.
[5] Applies only to Bandit Shooting with additional exploration mechanisms, see Section A.4.
[6] $c_{noise} = 1.0$ for STS Conv. and $c_{noise} = 0.3$ for STS MLP.
[7] The maximum number of nodes visited in a single planning pass, see Section A.5.
[8] If the last layer of a value function neural network was initialized to 0, see Section A.8.2.
[9] Indicates how training targets (`CALCULATE_TARGET` in Algorithm 2) are obtained. *rew2goT* and *treeT* corresponds "reward-to-go" and "tree action-values" described in Section A.1.

Table 3: Default values of hyper-parameters used in our experiments.

A.4 BANDIT SHOOTING

Algorithm 7 Bandit Shooting Planner with additional exploration mechanisms, requires exploration weight c_{puct}, action sampling temperature τ, noise weight c_{noise} and Dirichlet distribution parameter α

function SELECT(`state`)
  $s \leftarrow$ `state`
  $P(s,a) \leftarrow (1 - c_{noise})\pi_\phi(s,a) + c_{noise}D$
  $U(s,a) \leftarrow \sqrt{\sum_{a'} N(s,a')}/(1 + N(s,a))$
  $a \leftarrow \arg\max_a (Q(s,a) + c_{puct}P(s,a)U(s,a))$
  $s', r \leftarrow model.\text{STEP}(s,a)$
  return $(s,a,r),\ s'$

function EXPAND(`leaf`)
  `The same as in Algorithm 3.`
function UPDATE(`path`, `rollout`)
  `The same as in Algorithm 3.`
function CHOOSE_ACTION(s)
  $a \sim \text{softmax}\big(\frac{1}{\tau}\log N(s,\cdot)\big)$
  return a

Algorithm 7 describes Bandit Shooting with additional exploration mechanisms: mixing the policy with Dirichlet noise (as in Silver et al. (2018)) and action sampling with temperature τ in CHOOSE_ACTION(s). The noise variable D is sampled from the Dirchlet distribution $Dir(\alpha)$ each time when PLANNER is called (see also Algorithm 2).

A.5 MCTS

In our experiments, we used various implementations of MCTS. The reasons were two-fold. First, some implementation details fit better Sokoban and some GRF. Second, we wanted to check in various cases that the multi-step expansion is beneficial, see Section A.6.

In Sokoban experiments, we used the MCTS implementation similar to the one in Miłoś et al. (2019), containing a loop avoidance mechanism and transposition tables. The loop avoidance mechanism

alters SELECT and CHOOSE_ACTION (see Algorithm 5) so that the selected path does not contain repetitions of states. The transposition tables are a rather standard technique, which proposes to accumulate search statistics (i.e., W, N, Q) for states of the environment (rather than for the nodes of the search tree, as it happens in the standard case).

In GRF, we used our custom implementation of MCTS based on the one in Silver et al. (2017). It uses leaf evaluation with Q-function and policy networks. The Q-function is used to evaluate all children of a given node at once (instead of separately invoking value function $\mathbf{V}_\theta$ in UPDATE). The policy network is considered to be 'prior' for choosing actions, similarly as in SELECT in Algorithm 7. Dirichlet noise, parameterized by α and c_{noise}, is mixed with the prior in the root and action sampling with temperature τ is used to choose action on the real environment, similarly as in Bandit Shooting with additional exploration mechanisms in Section A.4. Additionally, we put a limit, depth limit, on the maximum number of nodes visited in a single STS pass.

A.6 STS

We tested STS with two MCTS setups described in Section A.5. In both the cases we observed substantial experimental improvements as reported in Section A.7 and Section A.8. This alone, in our view, provides enough evidence that the *multi-step expansion* is a useful method.

Apart from this, STS offers practical computational benefits, which are analyzed below.

A.6.1 Computational benefits of STS

We distinguish three types of computational costs in MCTS (see Algorithm 5):

1. Traversing down the search tree (performed in SELECT and EXPAND).
2. Backpropagation of values and counts update (handled by UPDATE).
3. Evaluation of heuristics (value network $\mathbf{V}_\theta$, or Q-function and policy as described in Section A.5)

In large GRF experiments, we found that it was the first cost that dominated the remaining two. The reason is that the cost of building a search tree is quadratic to its depth. The use of *multi-step expansion* significantly reduces this cost as several nodes are added during single tree traversal. In our case, these benefits allowed for much smoother experimenting with GRF and are, arguably, a step towards developing more efficient planners. We expect this might be practically useful (i.e., costs 1 and 2 are dominant) when the search size is large, or the heuristic evaluation is relatively cheap compared to the environment step. This is the case in some of our GRF experiments. The GRF simulator is rather complex and slower than small MLP networks.

The following simple lemma offers some theoretical analysis.

Lemma A.6.1. *Assume that STS and MCTS build the same tree $\mathcal{T}$, starting from the root state s_0. Denote the number of nodes in $\mathcal{T}$ as C and the number of nodes to be added at a single multi-step expansion of STS as H. Then the number of steps in $\mathcal{T}$ performed by STS will be lower compared to MCTS by a factor in $[\frac{h-1}{2}, h]$.*

Proof. Lets consider h consecutive nodes $s_1, \ldots, s_h$ in the search tree added in a single EXPAND step during STS search. In STS, the number of steps, C_{STS}, in the tree during SELECT and EXPAND is equal to $h+d$, where d is distance between s_0 and s_1 in $\mathcal{T}$. To add the same set of nodes during MCTS search, one need h separate calls to SELECT and EXPAND. The total number of steps performed is $C_{MCTS} = \sum_{k=0}^{h-1} d + k + 1 = hd + h\frac{h-1}{2}$. Clearly,

$$\frac{h-1}{2} C_{STS} \leq C_{MCTS} \leq h C_{STS}.$$

Similar calculation hold for the costs of backpropagation. □

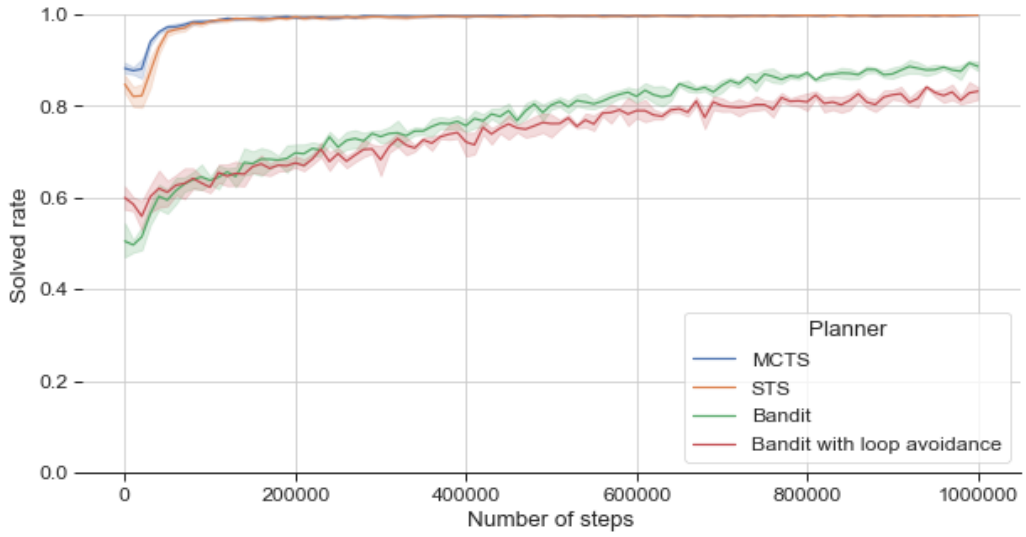


Figure 4: Sokoban on simpler boards: training curves for MCTS, STS and Bandit Shooting with and without loop avoidance. Mean over 5 seeds with shaded regions representing 95% confidence intervals.

A.7 SOKOBAN EXPERIMENTS

For a description of Sokoban see Section 4.1. In our experiments, we used inputs of dimension $(x, x, 7)$, where (x, x) is the size of the board ($(10, 10)$ in most cases) and 7 is one-hot encoding of the state of a given cell (enumerated as follows: wall, empty, target, box_target, box, player, player_target). In most experiments, we used 4 boxes. The agent is rewarded with 1 by putting a box into a designated spot and additionally with 10 when all the boxes are in place[1]. The action space consists of four movement directions (up, down, right, left).

A.7.1 EVALUATION EXPERIMENTS

In Table 4 we show full details of the evaluation experiment (which complements Table 1). Recall that in this experiment, we evaluated the planning capabilities of STS in isolation from training. To this end, we used a pre-trained value function and varied the number of passes C and the depth of multi-step expansion H, such that $H \cdot C$ remains constant. In Table 4, we present quantities (N_p, N_t, N_g), which measure planning costs for finding a solution (the average number of passes, tree nodes and game states observed, respectively, until the solution is found). We run experiments with and without the loop avoidance mechanism (see Section A.5). We observe that there is a sweet spot for the choice of H. It is evident for the 'no avoid loop' case, $C = 32, H = 8$. For this choice, the number of tree nodes, N_t, which is the most important metric, is the smallest. Interestingly, we observe a significant increase in the solved rate. This may be explained by the fact that the number of distinct visited game states, N_g, grows. This suggests that STS explores more aggressively and efficiently. For bigger H, we observe a further increase of the solved rate until some point, though at the cost of much bigger N_t.

In experiments with the avoid loop mechanism, there is a similar effect for $C = 64, H = 4$, though more subtle, probably because results are already quite strong. Moreover, we observe a more significant drop in performance as H increases (when planning resembles more shooting methods).

The values presented in Table 4 are averages over more than 5000 boards.

[1]Our Sokoban code is fully compatible with Racanière et al. (2017).

Scenario	C	H	S. rate	N_p	N_t	N_g
avoid loops	256	1	95.2%	1224	1224	716
	128	2	95.9%	569	1137	728
	64	4	96.5%	299	1194	830
	32	8	95.9%	173	1385	1040
	16	16	95.7%	114	1822	1333
	8	32	93.4%	79	2527	1528
	4	64	89%	62	3960	1491
	2	128	80%	52.7	6754	1207
no avoid loops	256	1	84.5%	1497	1497	376
	128	2	86.3%	724	1448	332
	64	4	87.8%	385	1541	370
	32	8	88.4%	185	1483	409
	16	16	89.5%	110	1754	539
	8	32	89.9%	84	2690	882
	4	64	85.2%	68	4463	1300
	2	128	65.3%	36	4589	967

Table 4: Evaluation of various STS settings on Sokoban

A.7.2 MCTS AND SHOOTING ON SIMPLER BOARDS

We found the Bandit Shooting method underperforming on Sokoban. As a sanity test, we tested a simpler setting with smaller boards of size $(6, 6)$ and two boxes. Learning curves are presented in Figure 4. MCTS and STS experiments quickly learn to solve over 99% of boards. Bandit Shooting experiment showed stable but much slower progress. We also evaluated the version of Bandit Shooting, with additional loop avoidance, see Section A.5. This mechanism was beneficial for MCTS and STS but failed to bring improvements for the shooting algorithms.

A.7.3 CORRECTING BIASED VALUE FUNCTIONS ESTIMATES WITH DEEP SEARCH

To generate value function heatmaps we evaluated the pre-trained MCTS value function for each possible agent position in a room. Figures 5 and 6 present two chosen rooms with their corresponding VF heatmaps. Specifically, the room in Figure 6 was solved by the STS with 10 passes and 5 steps of multi-step expansion and wasn't solved by the MCTS with 50 passes, both without the avoid loops mechanism. We include movies of both agents in this room in the code repository: `https://github.com/shoot-tree-search/sts/tree/master/movies`.

Because the value function is biased, it makes MCTS stuck in states with overestimated value. See Figure 6, in this room MCTS gets stuck in the bottom-left region. However, with a deeper tree, STS can get unstuck quicker and still find a solution. Remember that the search statistics (i.e., W, N, Q) are accumulated for states of the environment (see Appendix A.5). As this overestimated region gets searched deeper the bias in the value function gets discounted more and the agent figures out there are no rewards in reality. At some point, other actions will have a higher value and the agent has a chance to get unstuck and explore other parts of the room. That being said, it should be noted that this "potential well" will still attract the agent, make its planning paths distracted, even when it gets unstuck. STS is less vulnerable to this effect and is able to solve this room despite high bias in the value function.

A.8 GOOGLE RESEARCH FOOTBALL EXPERIMENTS

For a description of Google Research Football see Section 4.2. A Google Research Football academy environment is considered solved when an agent scores a goal. Reported results correspond to solved rates over 20 episodes in case of Shooting methods with an uniform and a pre-trained policy and around 30 episodes in case of all other methods. Results for MCTS, STS, and Shooting methods

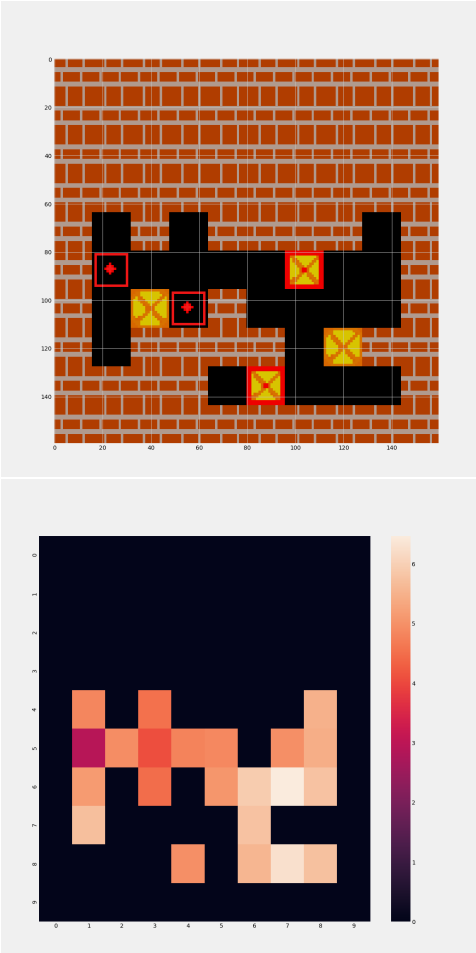


Figure 5: Sokoban value function heatmap, brighter means higher value estimate.

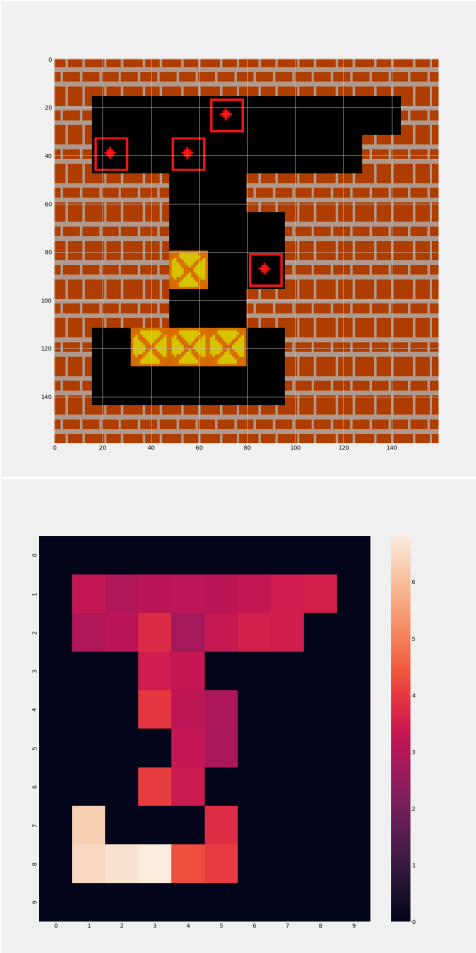


Figure 6: Sokoban value function heatmap, brighter means higher value estimate.

with the trained policy are medians of at least three training runs. During evaluations we disabled Dirichlet noise and action sampling (in Bandit Shooting Expl., MCTS and STS).

Google Research Football offers two major mode of observations: 'simple115' and 'extracted' (also called the super mini-map).

The simple115 state representation is consists of coordinates of players, players' movement directions, the ball position, a ball movement direction, a one-hot encoding of ball ownership, a one-hot encoding of which player is active. This totals in a vector of length 115.

The extracted state representation consists 4 stacked layers of size $(72, 96)$. Layers contain one-hot encoding of spatial positions of game entities. These are (on the subsequent layers): players on the left team, players on the right team, the ball, and the active player.

We note that even though the extracted representation contains 'less information' than simple115, it has been reported in Kurach et al. (2019) to generate better results.

In our experiments, we use the so-called checkpoint rewards, which provide an additional signal for approaching the goal area. Details can be found in Kurach et al. (2019), where they were introduced and used in large-scale experiments.

The action space in GRF consists of 19 actions representing high-level football behaviors (e.g. "Short Pass"), see Kurach et al. (2019, Table 1).

Figure 7 shows training curves for our STS agent on Google Research Football. Training was run for 3 days until convergence. On the y-axis is the solved rate calculated as described above in Section A.8. On the x-axis is the number of real steps in the environment (planning steps in the simulator are not added). Curves are mean over 3 training runs with different seeds and shaded regions represent 95% confidence intervals. Moreover, to smooth the curves, data points are averaged in the windows of 10000 steps.

A.8.1 SHOOTING METHODS

Tuning c_{puct} turned out to be the most important one to make Bandit Shooting work, see Algorithm 4. In a nutshell, it needs to be adjusted to scale of rewards (value function) in a given environment. In our experiments we found $c_{puct} = 2.5$ to work best.

Using additional Dirichlet noise, $c_{noise} > 0$, and action sampling on the real environment, $\tau > 0$ (see Algorithm 7) resulted in inferior results with an exception of the "Counterattack hard" scenario.

A.8.2 MCTS AND STS EXPERIMENTS

Apart from *multi-step expansion* we introduced another simple method, which might be of interest to the general public. Namely, before starting training, we set the weights of the last layer of the Q-value neural network to 0 (see Section A.3 for a detailed description of architectures). We observed that this significantly improved the training stability due to better exploration (and avoiding suboptimal strategies at the early stages of training). See 'No zero initialization' on Figure 12. This mechanism is similar to recent recommendations of (Andrychowicz et al., 2020, Section 3.2) given in the model-free setting.

A.9 ABLATIONS AND ANALYSIS

This section is devoted to analysis of various aspects of STS and comparisons to MCTS.

A.9.1 ANALYSIS OF THE TREE DEPTH

Recall that our hypothesis is that the benefits of *multi-step expansion* come from tilting the search towards DFS. While it is hard to formally prove this statement we were able to pin-point this effect in Sokoban and Google Football experiments, see Figure 8 and Figure 9 respectively.

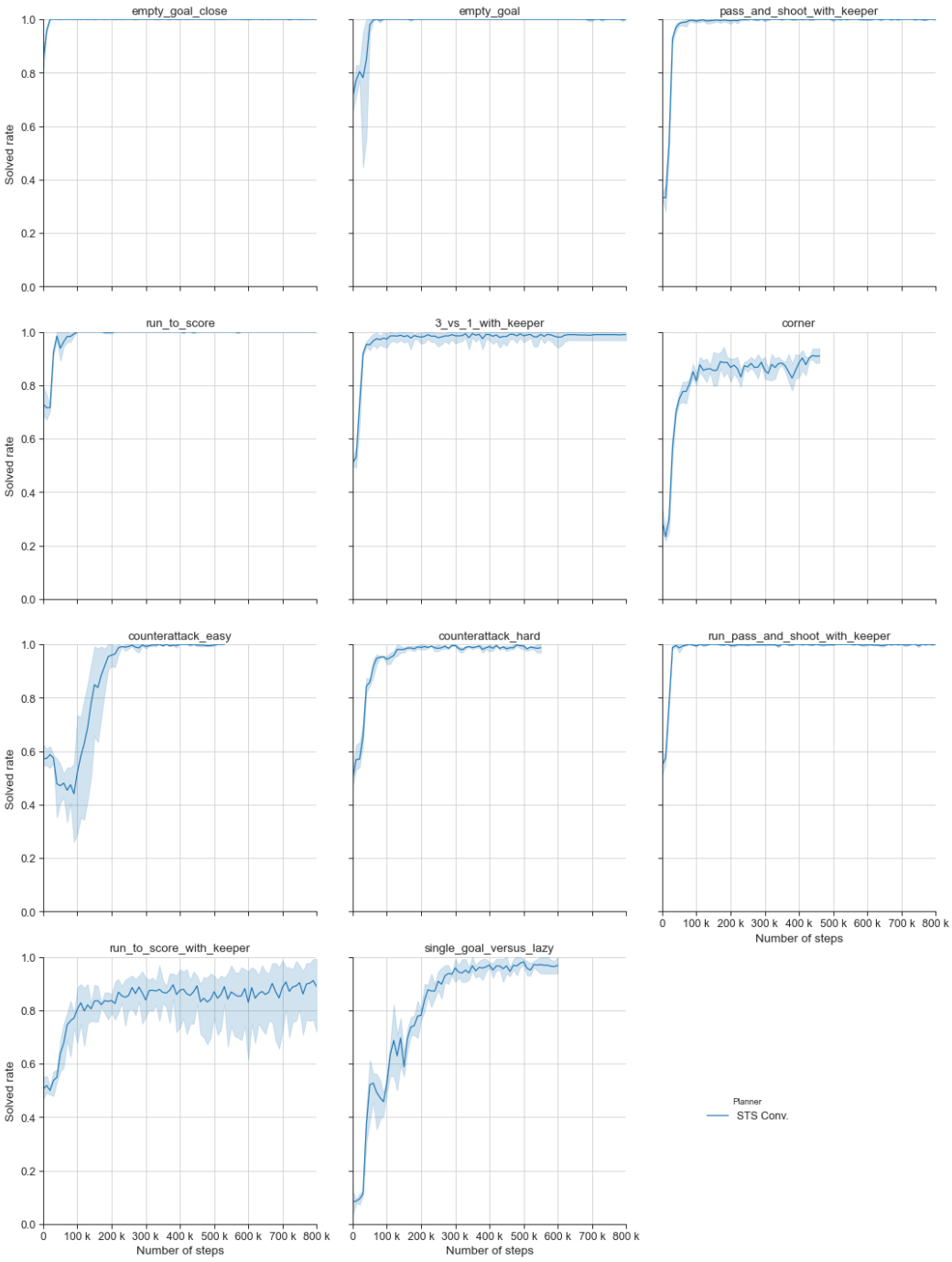


Figure 7: Google Research Football training curves for STS on GRF. Mean over 3 seeds with shaded regions representing 95% confidence intervals.

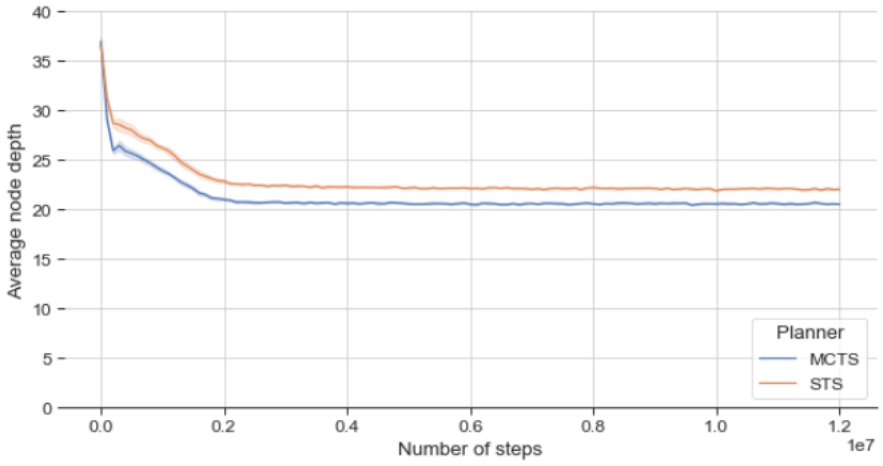


Figure 8: Search depth of MCTS and STS on Sokoban.

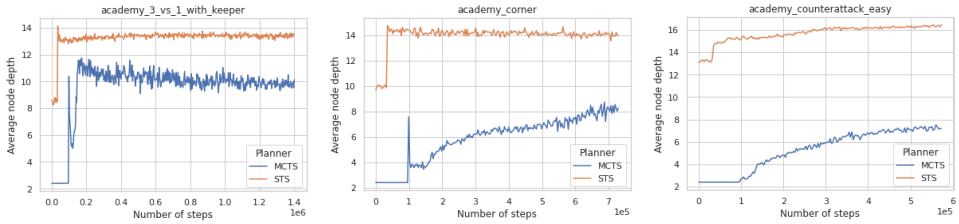


Figure 9: Search depth of MCTS and STS on three Football Academy tasks.

A.9.2 Comparison to AlphaGo

We run two additional experiments with rollout-based evaluation using different policies. These are meant to provide more evidence to support our hypothesis that the benefit of STS is due to the *multi-step expansion* mechanism.

In each experiment, the rollout was truncated after 10 steps to ensure fair comparison with STS. The return after the last step of the rollout was approximated using the Q-value network. This value and rewards collected were used to calculate the leaf's value in the same way as in AlphaGo. In experiments, we tested two strategies for generating rollouts:

1. Actions sampled from the prior policy - the same setup as in AlphaGo, except for rollout truncation and the choice of the policy. AlphaGo used a policy pretrained on expert data. Since we do not have access to such data for Google Football, we instead used the prior policy trained over the course of the algorithm.
2. Actions chosen deterministically, to maximize $Q(s,a) + c_{PUCT} * \pi(a|s)$. Q was computed by a neural network and π is the probability given by the trained prior distribution. We recall that this setup is equivalent to STS except for the crucial fact that STS adds the expanded leaves to the search tree.

We observed that strategy 1. performed very poorly, which highlights the importance of using neural networks for leaf evaluation. Strategy 2. performed significantly better but still worse than STS. We believe that these results strengthen the evidence that the advantage of STS stems from the algorithmic reasons by building a more efficient search tree. Note the tasks on which the two evaluated strategies performed the worst, i.e. counterattack_easy, counterattack_hard, single_goal_versus_lazy, are those with the longest lengths of a successful episode. This supports the argument that STS better handles problems with long planning horizons.

Method		3 vs. 1 with keeper	Corner	Counterattack easy	Counterattack hard	Empty goal	Empty goal close	Pass and shoot with keeper	Run pass and shoot with keeper	Run to score	Run to score with keeper	Single goal versus lazy
MCTS	Conv.	0.81	0.50	0.31	0.31	0.99	**1.00**	0.45	0.89	0.70	0.00	0.00
	Conv.+r.r.	0.91	0.09	0.00	0.03	0.00	**1.00**	**1.00**	**1.00**	0.00	0.06	0.00
	Conv.+d.r.	**1.00**	**0.81**	0.06	0.35	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.94	0.25
STS	Conv.	**1.00**	**0.81**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**0.97**	**0.97**

Table 5: Comparison of STS with leaf evaluation using a Q-value network and MCTS with different leaf evaluations: Q-value network only (Conv.), network + deterministic rollout (Conv.+d.r.), network + random rollout using the prior policy (Conv.+r.r.). In all experiments we used the same convolutional network architecture. The reported results are the median solved rates across 3 runs.

A.9.3 Ablations of STS design choices

The ablations were performed on three environments from GRF Academy: *corner*, *counterattack hard* and *empty goal*, see Figure 12. The first two environments are difficult, while the last one is easy. The following parameters or settings were subject to analysis (they correspond to the labels in Figure 12):

- `prior noise weight`: a weight in the mixture of Dirichlet noise and the prior.
- `depth limit`: the maximum number of nodes visited in a single STS pass.
- `sampling temperature`: temperature for sampling the actions on the real environment.
- `MCTS n_passes 300`: this corresponds the standard MCTS setting with $H = 1$ (MCTS) and $C = 300$
- `Value network n_passes`: value network is used instead of Q-function. Note that `n_passes` $= 2$ matches roughly the Q-function version in terms of visited states (recall, see Section 5, that Q-function evaluates all children at once and that number of actions in GRF is 19).
- `No policy`: instead of a learned policy network, a uniform policy is used.
- `No zero initialization`: the last layer of the value function neural network was not initialized to 0 (see description at the beginning of Section A.8.2).

The default setup (denoted as `Prior noise weight 0.1`) is always positioned at the top in Figure 12. It uses parameters described in Table 3 in the Google Research Football STS column.

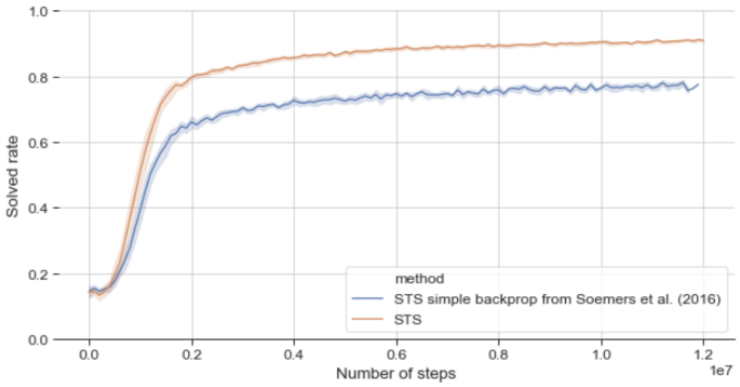
A.9.4 Ablation - backprop weight

This ablation aims to verify if there is benefit of the aggregate backprop implemented in `UPDATE` in Algorithm 6. We compare to the method proposed by Soemers et al. (2016), which backpropagates only the last value. We observed a clear advantage of STS on the Sokoban domain, see Figure A.9.4.

A.10 Multi-step expansion analysis on toy problems

First, consider an MDP presented at the top of Figure 11. It showcases the situation when the errors are systematic: in the vicinity of the starting state s_0, the estimates of the value function are biased (for simplicity set to 0 and shown as white vertices), while the values in the area surrounding terminal states are accurate (shown as color vertices). This example is an exaggeration. However, something similar can happen in practice, when information is propagated with TD-like methods or the environment has an "easy" region, which is hard to find. Under these circumstances, STS, given large enough H, will be able to reach accurate values (color vertices) within a few passes. On the contrary, MCTS would explore the whole uncertain area (white vertices) in a breadth-first fashion.

Second, consider an MDP shown at the bottom of Figure 11. It illustrates the case when the errors are "pseudo-random". In this MDP all rewards are 0 except the marked edges, where they are $-a, a > 0$.



Starting from s_0, the agent can move only to the right. The perfect value function is 0 in every state, however we assume that the current noisy value estimates equal to ϵ_i on the "tail" part of the diagram. In this example, we assume that the errors arise in interactions of many factors, thus can be modeled as i.i.d. centered random variables ϵ_i such that $\mathbb{E}|\epsilon_i| < +\infty$.

The optimal path, going over the green edge and later over the tail, is accompanied by several 'decoy' paths (marked in orange). They will not be entered unless errors on the tail have accumulated below $-a$. We denote the probability of such an event by p_H, where H is the number of steps in the multi-step expansion ($H = 1$ corresponds to MCTS). In Lemma A.10.1, we show that $p_1 > p_H$ for $H \geq 2$, and in fact $p_H \to 0$ when $H \to +\infty$.

Lemma A.10.1. *Under the above assumptions $p_1 > p_H$ and $p_H \to 0$.*

Proof. Assume that for the first $\ell \geq 2$ steps of the search tree was unfolded via the middle (green) edge and further via the tail. The state-action value estimated by the MCTS/STS is thus $q_\ell = (\epsilon_0 + \ldots + \epsilon_{\ell-2})/\ell$. Consequently,

$$p_H = \mathbb{P}(\exists_{k \in \mathbb{N}} q_{kH} < -a).$$

The claims follow from the fact $q_\ell \to 0$ a.s., which itself is the consequence of the strong law of large numbers. □

As the lemma serves mainly the illustrative purpose we used the i.i.d. assumption, which can be easily weakened. As a test we simulate the case $\epsilon_i \sim \mathcal{N}(0, 1)$ and $a = 0.3$. In this case $p_1 = 0.56, p_2 = 0.46, p_4 = 0.35, p_8 = 0.41, p_{16} = 0.21$. Note that p_1/p_H is as high as 3 for $H = 16$ and quite natural choice of a and ϵ_i.

A.10.1 High computational budgets

STS is predominantly meant to improve the search for modest computational budgets. When the budget gets bigger, any search becomes more exhaustive, and the benefits are likely to diminish. To our pleasant surprise, see Figure 10, we still observe certain advantages of STS in later phases of training, even though MCTS behaves better initially. This observation might suggest another interesting line of inquiry, namely, developing methods for adaptive breadth/depth balancing (e.g. via changing H).

A.11 Infrastructure used

We ran our experiments on clusters with servers typically equipped with 24 or 28 CPU cores and 64GB of memory. A typical experiment was 72 hours long (the timeout set on the clusters), which was enough for most experiments. Experiments that did not converge during this time were resumed.

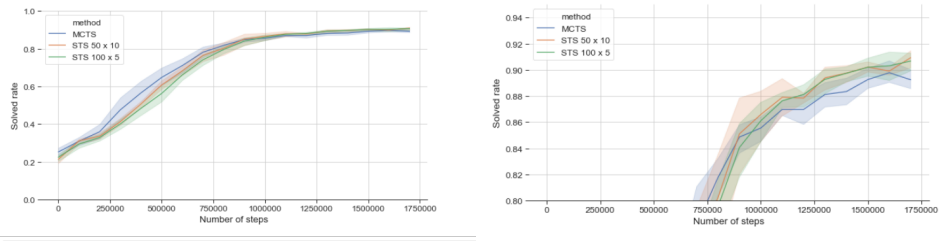


Figure 10: Experiment with big computational budget $C \cdot h = 500$ on Sokoban.

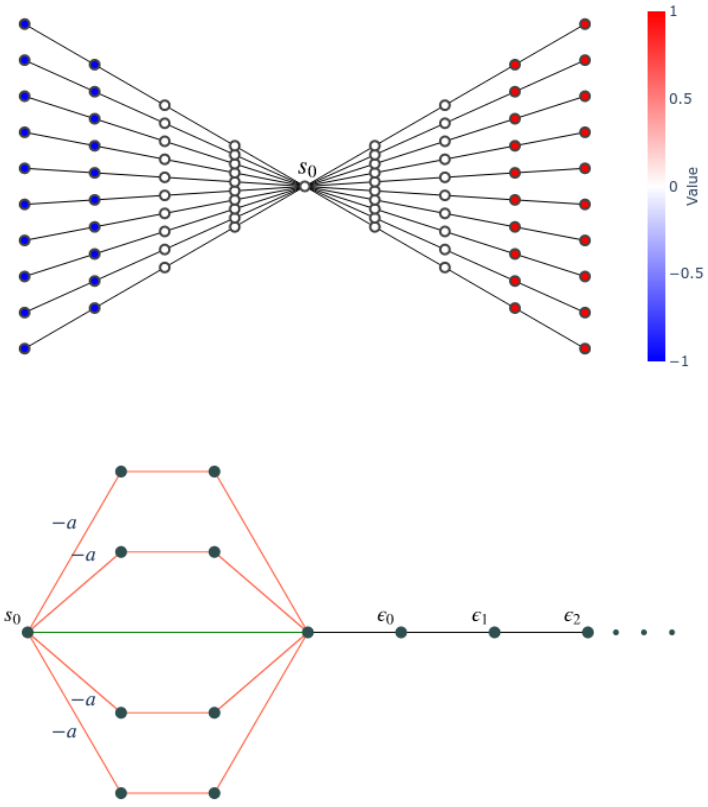


Figure 11: Visualization of the toy environments.

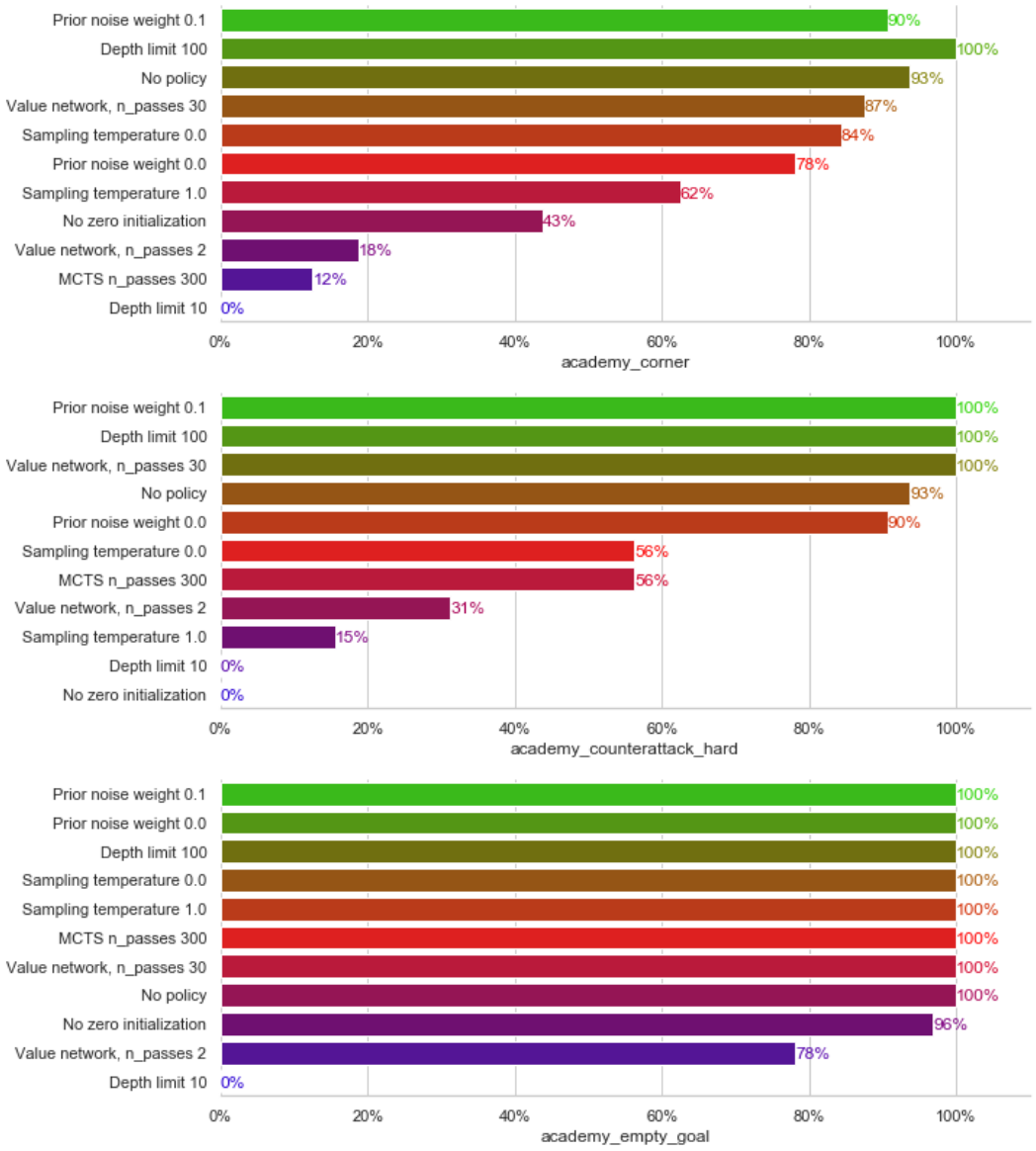


Figure 12: Ablations performed GRF Academy environments: *corner*, *counterattack hard* and *empty goal*.